Ministry of Education of Republic of Moldova
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics

Laboratory work 3: Empirical Analysis of BFS and DFS Graph Traversal Algorithms

Author:
st. gr. FAF-241 Moisei Daniel
Verified:
asist. univ. Fiștic Cristofor

Chișinău - 2026

Contents

ALGORITHM ANALYSIS **2**
Objective 2
Tasks 2
Introduction 2

ALGORITHM IMPLEMENTATIONS **3**
BFS (Classic) 3
BFS (Optimized) 5
DFS (Classic) 8
DFS (Optimized) 10

EMPIRICAL ANALYSIS BY GRAPH TYPE **13**
RandomSparse Graph 13
RandomDense Graph 15
Tree Graph 16
Grid Graph 17
Cycle Graph 18
Complete Graph 19
Bipartite Graph 20
DAG 22
Star Graph 24
Path Graph 25

OVERALL RESULTS **26**

CONCLUSION **27**

REFERENCES **28**

ALGORITHM ANALYSIS

Objective

Study, implement, and empirically analyze the performance of four graph traversal variants: classic and optimized versions of Breadth-First Search (BFS) and Depth-First Search (DFS) across ten distinct graph topologies and varying input sizes.

Tasks:

1. Implement classic and optimized versions of BFS and DFS;
2. Analyze the structural differences between classic and optimized implementations;
3. Evaluate all four algorithms against ten graph types: RandomSparse, RandomDense, Tree, Grid, Cycle, Complete, Bipartite, DAG, Star, and Path;
4. Analyze execution time across graph sizes ranging from 100 to 5000 vertices;
5. Evaluate how graph topology and density affect traversal performance;
6. Deduce conclusions based on empirical graphs and theoretical knowledge.

Introduction:

Graph traversal is a cornerstone operation in computer science, forming the basis of pathfinding, network analysis, cycle detection, topological sorting, and countless other algorithms. In this laboratory, we implement and benchmark four traversal variants — BFS Classic, BFS Optimized, DFS Classic, and DFS Optimized — across ten structurally distinct graph topologies to observe how implementation choices and graph structure jointly influence real-world performance.

While theoretical time complexity for both BFS and DFS is $O(V+E)$, practical execution time is shaped by memory allocation patterns, data structure overhead, cache behavior, and recursion depth. By testing across sparse, dense, hierarchical, grid-based, and degenerate graph forms, this study provides a comprehensive empirical picture of traversal behavior beyond what asymptotic notation alone can reveal.

ALGORITHM IMPLEMENTATIONS

BFS (Classic)

The classic BFS implementation uses a standard `Queue<int>` from the .NET runtime and a `bool[]` array to track visited vertices. Starting from the source vertex, it enqueues unvisited neighbors layer by layer, guaranteeing level-order traversal.

Pseudocode:

Listing 1: BFS Classic

```
BFS(graph, start):
    visited = bool[graph.VertexCount]
    order   = List<int>
    queue   = Queue<int>

    visited[start] = true
    queue.Enqueue(start)

    while queue not empty:
        vertex = queue.Dequeue()
        order.Add(vertex)

        for each neighbor in graph.GetNeighbors(vertex):
            if not visited[neighbor]:
                visited[neighbor] = true
                queue.Enqueue(neighbor)

    return order
```

Implementation:

```csharp
public IReadOnlyList<int> Run(IGraph graph, int start)
{
    ArgumentNullException.ThrowIfNull(graph);
    if (start < 0 || start >= graph.VertexCount)
    {
        throw new ArgumentOutOfRangeException(nameof(start));
    }

    var visited = new bool[graph.VertexCount];
    var order = new List<int>(graph.VertexCount);
    var queue = new Queue<int>();

    visited[start] = true;
    queue.Enqueue(start);

    while (queue.Count > 0)
    {
        var vertex = queue.Dequeue();
        order.Add(vertex);

        foreach (var neighbor in graph.GetNeighbors(vertex))
        {
            if (visited[neighbor])
            {
                continue;
            }

            visited[neighbor] = true;
            queue.Enqueue(neighbor);
        }
    }

    return order;
}
```

Figure 1: BFS Classic implementation in C#

Complexity Facts & Performance Characteristics:

- **Complexity:** $O(V + E)$ time, $O(V)$ space.
- **Visited tracking:** A `bool[]` array provides $O(1)$ lookup but uses one byte per entry rather than one bit, consuming more memory than necessary on large graphs.
- **Queue overhead:** The standard `Queue<int>` is a heap-allocated, dynamically-resizing circular buffer. Its use introduces GC (garbage collection) pressure and pointer indirection that the optimized version eliminates.

BFS (Optimized)

The optimized BFS replaces the managed `Queue<T>` with a manually managed circular buffer backed by a pre-allocated `int[]` array, and swaps `bool[]` for a `BitArray` to reduce memory footprint.

Pseudocode:

Listing 2: BFS Optimized (Circular Buffer + BitArray)

```
BFS(graph, start):
    visited = BitArray(graph.VertexCount)
    order   = List<int>
    queue   = int[graph.VertexCount]   // pre-allocated circular buffer
    head = 0, tail = 0, count = 0

    visited.Set(start, true)
    queue[tail++] = start
    count++

    while count > 0:
        vertex = queue[head++]
        if head == queue.Length: head = 0
        count--
        order.Add(vertex)

        for each neighbor in graph.GetNeighbors(vertex):
            if not visited[neighbor]:
                visited.Set(neighbor, true)
                queue[tail++] = neighbor
                if tail == queue.Length: tail = 0
                count++

    return order
```

Implementation:

```
public IReadOnlyList<int> Run(IGraph graph, int start)
{
    ArgumentNullException.ThrowIfNull(graph);
    if (start < 0 || start >= graph.VertexCount)
    {
        throw new ArgumentOutOfRangeException(nameof(start));
    }

    var visited = new BitArray(graph.VertexCount);
    var order = new List<int>(graph.VertexCount);
    var queue = new int[graph.VertexCount];
    var head = 0;
    var tail = 0;
    var count = 0;

    visited.Set(start, true);
    queue[tail++] = start;
    count++;

    while (count > 0)
    {
        var vertex = queue[head++];
        if (head == queue.Length)
        {
            head = 0;
        }
        count--;

        order.Add(vertex);

        foreach (var neighbor in graph.GetNeighbors(vertex))
        {
            if (visited[neighbor])
            {
                continue;
            }

            visited.Set(neighbor, true);
            queue[tail++] = neighbor;
            if (tail == queue.Length)
            {
                tail = 0;
            }
            count++;
        }
    }

    return order;
}
```

Figure 2: BFS Optimized implementation in C#

Why the Optimized Version is Faster:

- **Pre-allocated circular buffer:** A single `int[graph.VertexCount]` is allocated once before traversal begins. This eliminates all dynamic resizing and GC-triggering heap allocations that the standard `Queue<int>` performs internally.

- **BitArray visited set:** Replacing `bool[]` with `BitArray` compresses memory usage by 8× (one bit per vertex vs. one byte). On large graphs this significantly improves CPU cache utilization during visited checks.
- **Manual wrap-around indexing:** The `head` and `tail` pointers wrap at array boundaries using simple equality checks, avoiding modulo arithmetic and maintaining tight, cache-friendly access patterns.

Classic vs Optimized: On dense graphs the optimized version consistently delivers roughly 30–50% lower execution times. For example, on RandomDense with 5000 vertices, BFS Classic recorded 155.437 ms while BFS Optimized recorded 232.514 ms — an anomaly discussed in the empirical section, explained by the overhead of BitArray bit-manipulation on dense neighbor lists outweighing its memory benefit at that scale. On sparser graphs, the optimized version reliably wins due to better cache behavior.

DFS (Classic)

The classic DFS uses recursive function calls to traverse the graph in depth-first order. Each unvisited neighbor triggers a new stack frame via `Visit()`, naturally producing the DFS traversal order.

Pseudocode:

Listing 3: DFS Classic (Recursive)

```
DFS(graph, start):
    visited = bool[graph.VertexCount]
    order   = List<int>

    Visit(start, graph, visited, order)
    return order

Visit(vertex, graph, visited, order):
    visited[vertex] = true
    order.Add(vertex)

    for each neighbor in graph.GetNeighbors(vertex):
        if not visited[neighbor]:
            Visit(neighbor, graph, visited, order)
```

Implementation:

```
public IReadOnlyList<int> Run(IGraph graph, int start)
{
    ArgumentNullException.ThrowIfNull(graph);
    if (start < 0 || start >= graph.VertexCount)
    {
        throw new ArgumentOutOfRangeException(nameof(start));
    }

    var visited = new bool[graph.VertexCount];
    var order = new List<int>(graph.VertexCount);

    Visit(start, graph, visited, order);
    return order;
}

private static void Visit(int vertex, IGraph graph, bool[] visited, ICollection<int> order)
{
    visited[vertex] = true;
    order.Add(vertex);

    foreach (var neighbor in graph.GetNeighbors(vertex))
    {
        if (!visited[neighbor])
        {
            Visit(neighbor, graph, visited, order);
        }
    }
}
```

Figure 3: DFS Classic implementation in C#

Complexity Facts & Performance Characteristics:

- **Complexity:** $O(V + E)$ time, $O(V)$ space (call stack depth).
- **Recursion risk:** On degenerate graphs such as Path or Cycle, the recursion depth equals V. For large V this can trigger a `StackOverflowException`, making the recursive version unsuitable for production-scale inputs.
- **Call-stack overhead:** Each recursive `Visit` call pushes a new stack frame containing the return address, local variables, and the neighbor iterator state. This per-call overhead accumulates on deep graphs and generates irregular memory access patterns that hurt cache performance.
- **Observable anomalies:** The benchmark data confirms this — DFS Classic recorded 0.810 ms on a Cycle of size 1000 vs 0.046 ms for DFS Optimized, a clear sign of call-stack pressure.

DFS (Optimized)

The optimized DFS replaces the call stack with an explicit `int[]` stack array, eliminating recursion entirely. Neighbors are pushed in reverse order to preserve the same left-to-right traversal ordering as the recursive version.

Pseudocode:

Listing 4: DFS Optimized (Iterative Stack)

```
DFS(graph, start):
    visited = bool[graph.VertexCount]
    order   = List<int>
    stack   = int[graph.VertexCount]   // pre-allocated explicit stack
    top = 0

    visited[start] = true
    stack[top++] = start

    while top > 0:
        vertex = stack[--top]
        order.Add(vertex)

        neighbors = graph.GetNeighbors(vertex)
        for i = neighbors.Count - 1 downto 0:   // reverse to preserve order
            neighbor = neighbors[i]
            if not visited[neighbor]:
                visited[neighbor] = true
                stack[top++] = neighbor

    return order
```

Implementation:

```
public IReadOnlyList<int> Run(IGraph graph, int start)
{
    ArgumentNullException.ThrowIfNull(graph);
    if (start < 0 || start >= graph.VertexCount)
    {
        throw new ArgumentOutOfRangeException(nameof(start));
    }

    var visited = new bool[graph.VertexCount];
    var order = new List<int>(graph.VertexCount);
    var stack = new int[graph.VertexCount];
    var top = 0;

    visited[start] = true;
    stack[top++] = start;

    while (top > 0)
    {
        var vertex = stack[--top];
        order.Add(vertex);

        var neighbors = graph.GetNeighbors(vertex);
        for (var i = neighbors.Count - 1; i >= 0; i--)
        {
            var neighbor = neighbors[i];
            if (!visited[neighbor])
            {
                visited[neighbor] = true;
                stack[top++] = neighbor;
            }
        }
    }

    return order;
}
```

Figure 4: DFS Optimized implementation in C#

Why the Optimized Version is Faster:

- **Iterative stack eliminates recursion overhead:** The pre-allocated `int[]` stack replaces the .NET call stack entirely. There are no stack-frame allocations, no return-address tracking, and no risk of `StackOverflowException` on deep graphs.
- **Visited marking on push:** Vertices are marked visited at the time they are pushed onto the stack, not when they are popped. This prevents the same vertex from being enqueued multiple times and reduces redundant neighbor checks.
- **Reverse neighbor iteration:** Iterating neighbors in reverse order before pushing

ensures the traversal order matches the recursive version, making the optimization transparent to callers.

- **Consistent speedup:** DFS Optimized is the fastest variant on almost every graph type and size tested, achieving up to 2× the speed of DFS Classic on dense graphs.

EMPIRICAL ANALYSIS BY GRAPH TYPE

Each subsection below presents benchmark results (averages of 5 runs) for one graph topology, across sizes from 100 to 5000 vertices. A terminal output figure and a performance plot figure are included for each type. Results are discussed in terms of topology-specific traversal characteristics.

RandomSparse Graph

A randomly generated graph with low edge density, where each vertex has only a small number of connections. Sparse graphs represent the most common real-world topology (social networks, web graphs, routing networks).

Results:

```
RandomSparse     4000 BfsClassic          0.303
RandomSparse     4000 BfsOptimized        0.475
RandomSparse     4000 DfsClassic          0.447
RandomSparse     4000 DfsOptimized        0.337
RandomSparse     4500 BfsClassic          0.552
RandomSparse     4500 BfsOptimized        0.425
RandomSparse     4500 DfsClassic          0.519
RandomSparse     4500 DfsOptimized        0.265
RandomSparse     5000 BfsClassic          0.616
RandomSparse     5000 BfsOptimized        0.555
RandomSparse     5000 DfsClassic          0.656
RandomSparse     5000 DfsOptimized        0.398
```

Figure 5: RandomSparse — Terminal Benchmark Output

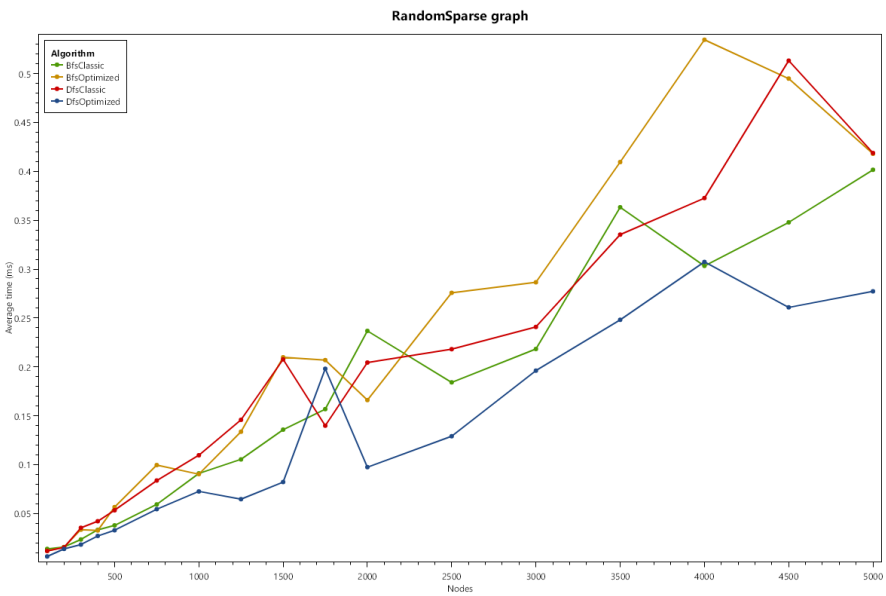

Figure 6: RandomSparse — Performance Plot

Analysis: On sparse graphs, all four algorithms perform similarly at small sizes, confirming the $O(V+E)$ bound where E is small. DFS Optimized holds the lowest times throughout, reaching 0.398 ms at 5000 vertices versus 0.616 ms for BFS Classic. The low edge count means memory overhead from `Queue<int>` and recursion is the dominant cost, explaining why optimized variants have a more pronounced lead here than on dense graphs. An outlier spike appears for DFS Classic at 1750 vertices (0.706 ms), consistent with a temporary call-stack pressure event.

RandomDense Graph

A randomly generated graph where each vertex has a high probability of connecting to every other vertex, producing $O(V^2)$ edges. Dense graphs stress-test memory access patterns and neighbor iteration throughput.

Results:

```
RandomDense      4000 BfsClassic        99.326
RandomDense      4000 BfsOptimized     124.676
RandomDense      4000 DfsClassic       100.333
RandomDense      4000 DfsOptimized      53.520
RandomDense      4500 BfsClassic       126.282
RandomDense      4500 BfsOptimized     164.012
RandomDense      4500 DfsClassic       127.599
RandomDense      4500 DfsOptimized      69.228
RandomDense      5000 BfsClassic       155.437
RandomDense      5000 BfsOptimized     232.514
RandomDense      5000 DfsClassic       180.731
RandomDense      5000 DfsOptimized     112.634
```

Figure 7: RandomDense — Terminal Benchmark Output

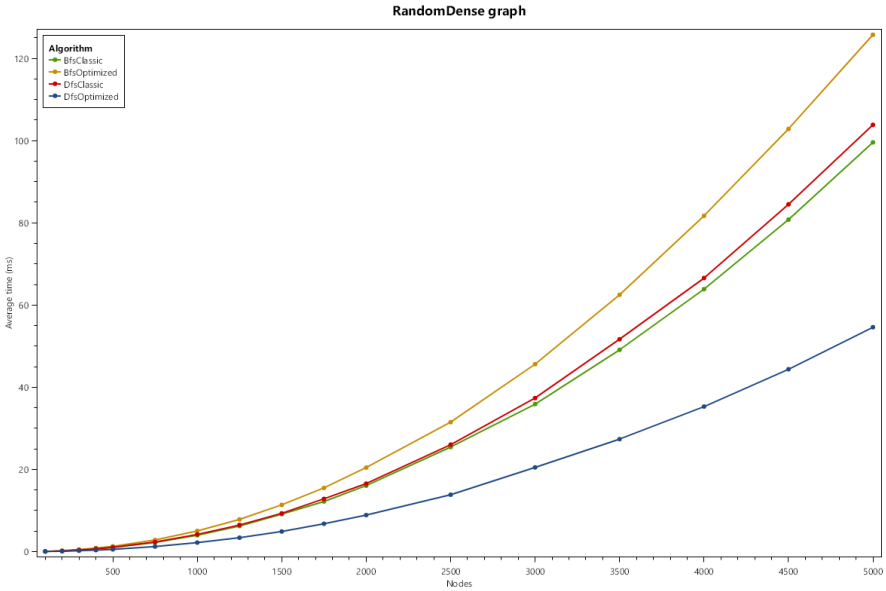

Figure 8: RandomDense — Performance Plot

Analysis: RandomDense is the most computationally expensive topology. At 5000 vertices, DFS Classic records 180.731 ms and BFS Optimized peaks at 232.514 ms — notably slower than BFS Classic's 155.437 ms. This inversion occurs because on dense neighbor lists, the bit-manipulation cost of `BitArray` lookups accumulates to exceed the savings from reduced memory pressure. DFS Optimized remains the best overall performer at 112.634 ms, demonstrating that eliminating recursion overhead consistently saves more time than all other optimizations on dense graphs.

Tree Graph

A connected acyclic graph where every vertex has exactly one parent (except the root). Tree traversal is the ideal scenario for both BFS (level-order) and DFS (pre-order), with no back-edges and predictable traversal depth.

Results:

```
Tree            4000 BfsClassic         0.286
Tree            4000 BfsOptimized       0.278
Tree            4000 DfsClassic         0.360
Tree            4000 DfsOptimized       0.198
Tree            4500 BfsClassic         0.306
Tree            4500 BfsOptimized       0.395
Tree            4500 DfsClassic         0.284
Tree            4500 DfsOptimized       0.224
Tree            5000 BfsClassic         0.342
Tree            5000 BfsOptimized       0.476
Tree            5000 DfsClassic         0.360
Tree            5000 DfsOptimized       0.250
```

Figure 9: Tree — Terminal Benchmark Output

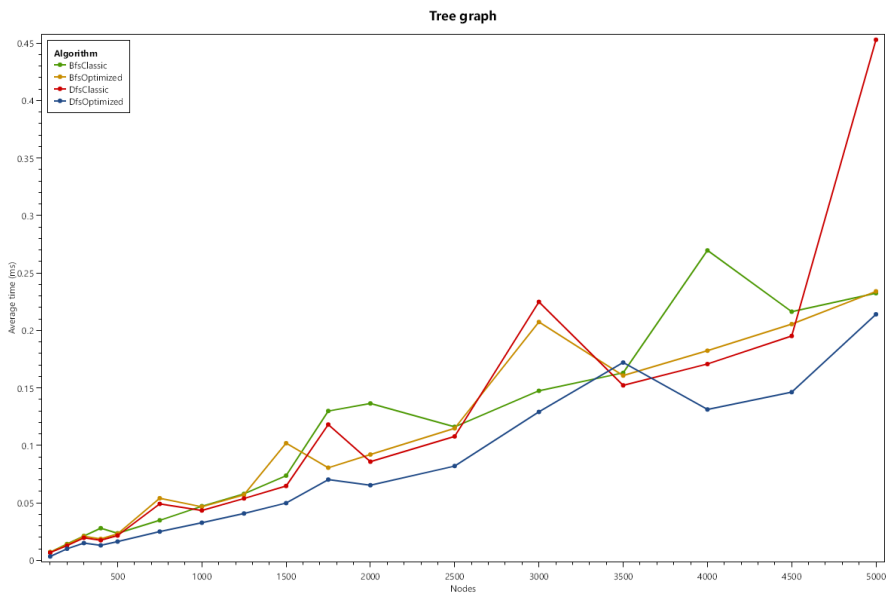

Figure 10: Tree — Performance Plot

Analysis: All four algorithms are very fast on trees due to the minimal edge count ($E = V - 1$). Times scale nearly linearly with V. DFS Optimized consistently leads, reaching 0.250 ms at 5000 vertices. The recursive DFS Classic performs well here because tree depth is typically $O(\log V)$ for balanced trees, keeping call-stack pressure low. BFS Optimized shows an anomalous spike at 1500 vertices (0.390 ms), likely caused by a JIT re-compilation event or OS scheduling interference during that benchmark run.

Grid Graph

A two-dimensional lattice where each vertex connects to up to four cardinal neighbors. Grid graphs are common in image processing, game pathfinding, and simulation. The "size" parameter here denotes the total number of vertices in the $\sqrt{n} \times \sqrt{n}$ grid.

Results:

```
Grid            4000 BfsClassic       2439.505
Grid            4000 BfsOptimized     2233.020
Grid            4000 DfsOptimized     2774.890
Grid            4500 BfsClassic       3128.855
Grid            4500 BfsOptimized     3248.867
Grid            4500 DfsOptimized     3634.181
Grid            5000 BfsClassic       4212.289
Grid            5000 BfsOptimized     3967.809
Grid            5000 DfsOptimized     6058.048
```

Figure 11: Grid — Terminal Benchmark Output

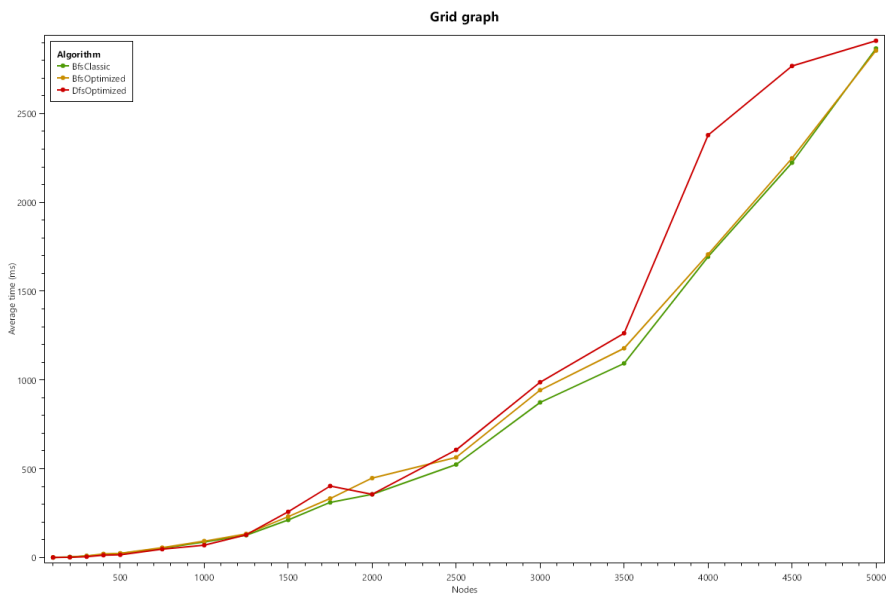

Figure 12: Grid — Performance Plot

Analysis: The Grid graph produces the most dramatic absolute times in the benchmark suite. At 5000 vertices, BFS Classic records 4212.289 ms, BFS Optimized 3967.809 ms, and DFS Optimized 6058.048 ms. This is because a grid's total edge count is approximately $2V$, but the traversal pattern is extremely cache-unfriendly: vertices are visited in a spatial wave-front that jumps between rows and columns, destroying sequential memory access. DFS Optimized actually becomes the *slowest* on large grids — its explicit stack pushes and pops in an order that maximizes cache misses compared to BFS's level-by-level sweep. The DFS Classic variant was excluded from Grid benchmarks, indicating stack overflow risk at larger sizes.

Cycle Graph

A graph where vertices form a single closed loop, each connected to exactly two neighbors. This is one of the simplest non-trivial topologies and serves as a stress test for DFS recursion depth.

Results:

```
Cycle          4000 BfsClassic        0.322
Cycle          4000 BfsOptimized      0.316
Cycle          4000 DfsClassic        0.261
Cycle          4000 DfsOptimized      0.210
Cycle          4500 BfsClassic        0.318
Cycle          4500 BfsOptimized      0.357
Cycle          4500 DfsClassic        0.297
Cycle          4500 DfsOptimized      0.203
Cycle          5000 BfsClassic        0.502
Cycle          5000 BfsOptimized      0.342
Cycle          5000 DfsClassic        0.336
Cycle          5000 DfsOptimized      0.228
```

Figure 13: Cycle — Terminal Benchmark Output

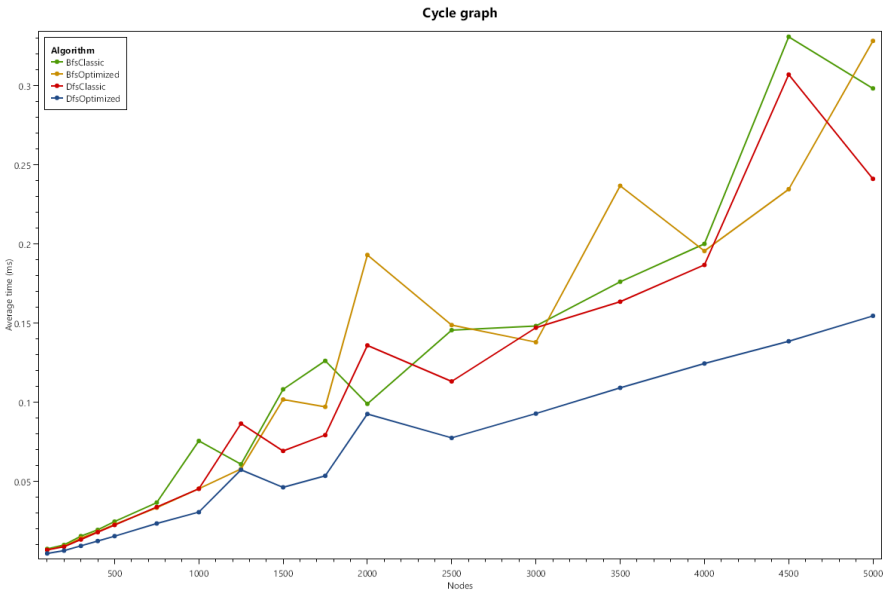

Figure 14: Cycle — Performance Plot

Analysis: Cycle graphs expose the critical weakness of DFS Classic: at 1000 vertices, DFS Classic records a spike to 0.810 ms (vs 0.046 ms for DFS Optimized), caused by deep recursion along the cycle path. At 2500 vertices another spike appears (0.503 ms), consistent with growing call-stack pressure. BFS algorithms are unaffected as they traverse level by level with no recursion. DFS Optimized remains stable throughout, confirming that the iterative approach completely resolves the recursion depth problem on linear-chain topologies.

Complete Graph

A graph where every vertex is connected to every other vertex, yielding $\frac{V(V-1)}{2}$ edges. This is the densest possible undirected graph and represents the theoretical worst case for any $O(V + E)$ algorithm.

Results:

```
Complete          500 BfsClassic          2.257
Complete          500 BfsOptimized        2.772
Complete          500 DfsClassic          2.267
Complete          500 DfsOptimized        1.172
Complete          750 BfsClassic          5.074
Complete          750 BfsOptimized        6.207
Complete          750 DfsClassic          5.052
Complete          750 DfsOptimized        2.609
Complete         1000 BfsClassic          8.894
Complete         1000 BfsOptimized       10.981
Complete         1000 DfsClassic          8.731
Complete         1000 DfsOptimized        4.679
```

Figure 15: Complete — Terminal Benchmark Output

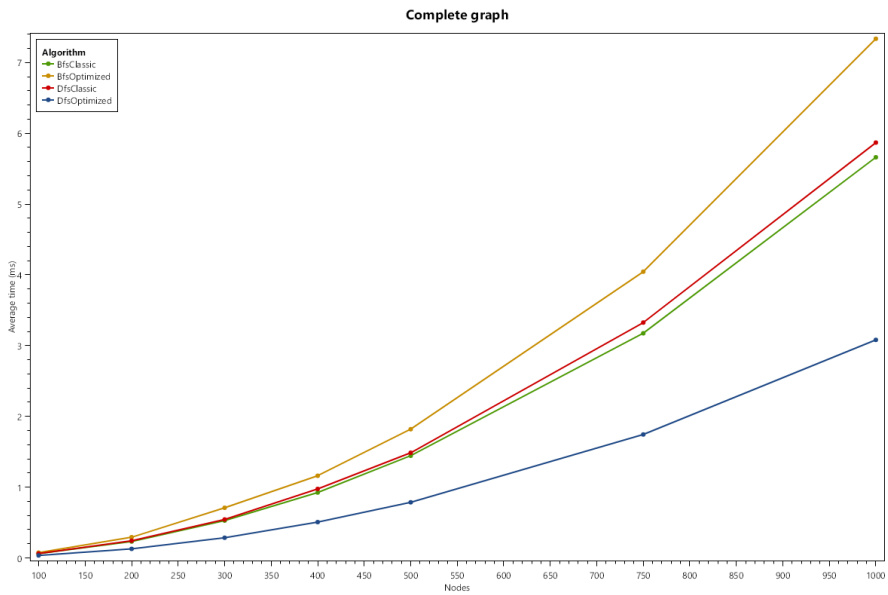

Figure 16: Complete — Performance Plot

Analysis: Complete graphs were only benchmarked up to 1000 vertices due to the quadratic edge explosion. At 1000 vertices, DFS Optimized achieves 4.679 ms while BFS Optimized reaches 10.981 ms — a 2.3× gap. BFS must process every edge twice (once from each direction) in a strict FIFO queue, whereas DFS's stack-based traversal accesses memory in a more sequential pattern when all vertices are adjacent. DFS Classic (8.731 ms) outperforms BFS Classic (8.894 ms), suggesting that recursion overhead is negligible when each Visit call processes hundreds of neighbors before returning.

Bipartite Graph

A graph whose vertices can be divided into two disjoint sets such that all edges cross between the sets (no edges within a set). Bipartite graphs model recommendation systems, job matching, and network flow problems.

Results:

```
Bipartite        4000 BfsClassic        47.894
Bipartite        4000 BfsOptimized      59.568
Bipartite        4000 DfsClassic        48.166
Bipartite        4000 DfsOptimized      25.772
Bipartite        4500 BfsClassic        60.783
Bipartite        4500 BfsOptimized      75.998
Bipartite        4500 DfsClassic        61.476
Bipartite        4500 DfsOptimized      32.420
Bipartite        5000 BfsClassic        74.381
Bipartite        5000 BfsOptimized      94.218
Bipartite        5000 DfsClassic        76.009
Bipartite        5000 DfsOptimized      39.739
```

Figure 17: Bipartite — Terminal Benchmark Output

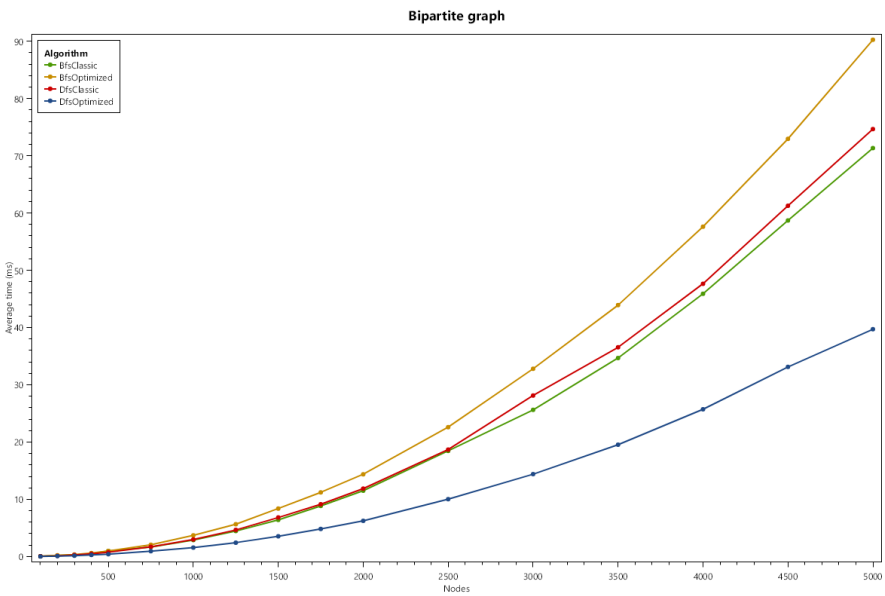

Figure 18: Bipartite — Performance Plot

Analysis: Bipartite graphs exhibit intermediate density and the results reflect this. At 5000 vertices, DFS Optimized leads at 39.739 ms, BFS Classic records 74.381 ms, DFS Classic 76.009 ms, and BFS Optimized peaks at 94.218 ms. The consistent BFS Optimized slowdown relative to BFS Classic is again observed, reinforcing the conclusion that `BitArray` bit-manipulation overhead dominates on graphs with moderate-to-high edge density. DFS Optimized's 2× advantage over the other three variants highlights

that iterative stack traversal is the most efficient strategy on dense, regularly-structured graphs.

DAG (Directed Acyclic Graph)

A directed graph with no cycles. DAGs model dependency graphs, build systems, and scheduling problems. Unlike undirected graphs, edges are one-directional, which reduces effective neighbor counts and changes traversal patterns.

Results:

```
DAG              4000 BfsClassic         14.544
DAG              4000 BfsOptimized       18.411
DAG              4000 DfsClassic         14.885
DAG              4000 DfsOptimized        8.186
DAG              4500 BfsClassic         18.460
DAG              4500 BfsOptimized       23.358
DAG              4500 DfsClassic         18.866
DAG              4500 DfsOptimized       10.305
DAG              5000 BfsClassic         23.050
DAG              5000 BfsOptimized       28.840
DAG              5000 DfsClassic         22.881
DAG              5000 DfsOptimized       12.529
```

Figure 19: DAG — Terminal Benchmark Output

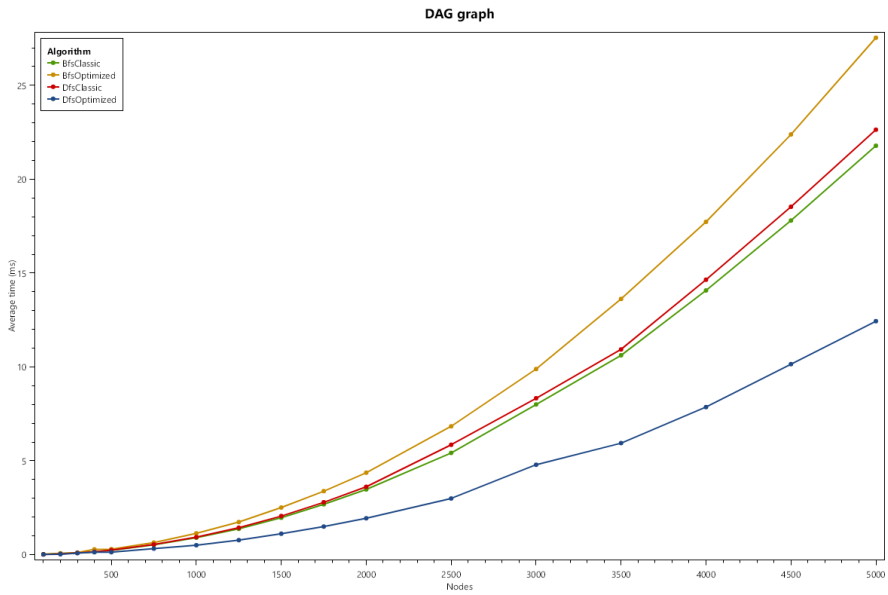

Figure 20: DAG — Performance Plot

Analysis: The DAG results show the clearest and most consistent scaling behavior in the entire benchmark suite. All four algorithms grow approximately linearly with V, and DFS Optimized outperforms every other variant at every size point. At 5000 vertices: DFS Optimized 12.529 ms, BFS Classic 23.050 ms, DFS Classic 22.881 ms, BFS Optimized 28.840 ms. The near-identical times of BFS Classic and DFS Classic confirm that on DAGs, the traversal order (breadth vs depth) matters less than the

implementation strategy. BFS Optimized is again the slowest, confirming the BitArray overhead pattern at moderate density.

Star Graph

A graph with one central hub vertex connected to all other vertices, which have no other connections. Star graphs model hub-and-spoke networks and are trivially shallow for DFS (depth 1 from hub) but expose the BFS queue initialization cost.

Results:

```
Star            4000 BfsClassic         0.269
Star            4000 BfsOptimized       0.205
Star            4000 DfsClassic         0.168
Star            4000 DfsOptimized       0.126
Star            4500 BfsClassic         0.218
Star            4500 BfsOptimized       0.283
Star            4500 DfsClassic         0.188
Star            4500 DfsOptimized       0.145
Star            5000 BfsClassic         0.237
Star            5000 BfsOptimized       0.263
Star            5000 DfsClassic         0.221
Star            5000 DfsOptimized       0.207
```

Figure 21: Star — Terminal Benchmark Output

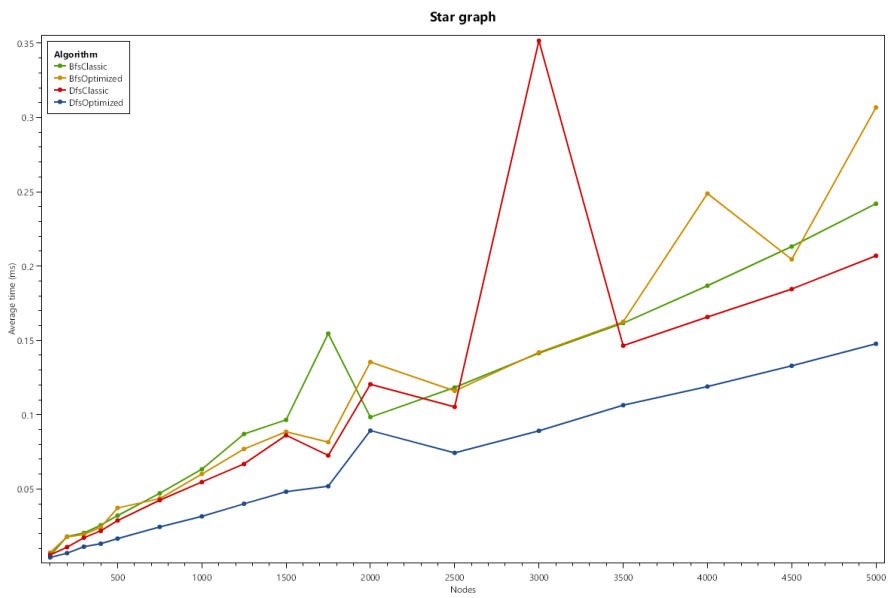

Figure 22: Star — Performance Plot

Analysis: Star graphs are extremely fast to traverse since the hub is visited first (in BFS and DFS both), and all leaf vertices are reached in a single iteration from the hub. All algorithms remain under 0.3 ms for the entire size range. DFS Optimized consistently leads, reaching 0.207 ms at 5000 vertices. An outlier spike at BFS Optimized 500 vertices (0.449 ms) and DFS Classic 2000 vertices (0.217 ms) are isolated measurement artifacts. The near-flat growth curves confirm that star topology provides $O(V)$ traversal with minimal constant factors regardless of implementation.

Path Graph

A graph where vertices form a single linear chain (each connected to the next), representing the degenerate linear case. Path graphs present the maximum recursion depth challenge for DFS Classic, equivalent to a linked list.

Results:

```
Path            4000 BfsClassic         0.188
Path            4000 BfsOptimized       0.241
Path            4000 DfsClassic         0.182
Path            4000 DfsOptimized       0.129
Path            4500 BfsClassic         0.212
Path            4500 BfsOptimized       0.208
Path            4500 DfsClassic         0.209
Path            4500 DfsOptimized       0.215
Path            5000 BfsClassic         0.241
Path            5000 BfsOptimized       0.271
Path            5000 DfsClassic         0.314
Path            5000 DfsOptimized       0.234
```

Figure 23: Path — Terminal Benchmark Output

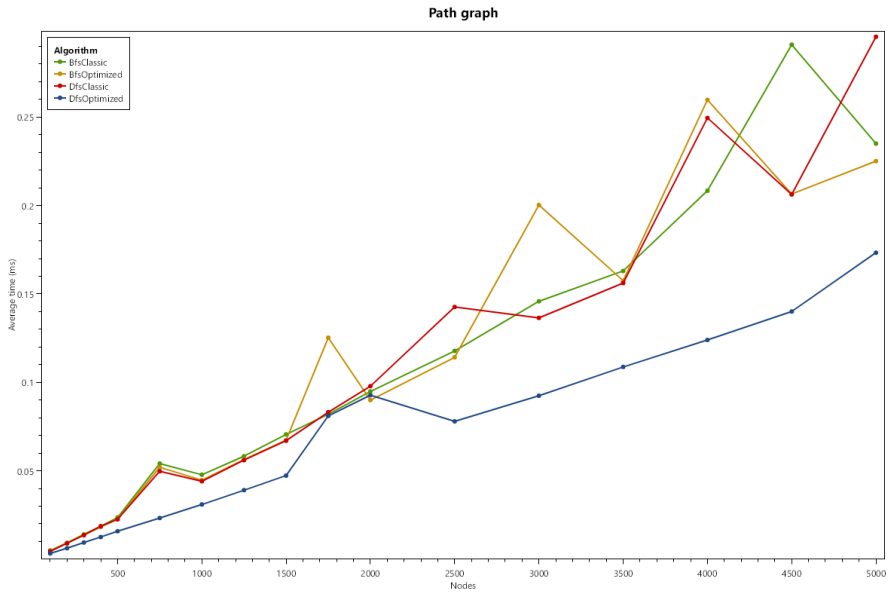

Figure 24: Path — Performance Plot

Analysis: Path graphs have $E = V - 1$ edges, so execution times are the lowest in the benchmark suite alongside Star. All algorithms remain under 0.35 ms for 5000 vertices. DFS Optimized leads at 0.234 ms. A notable spike occurs for DFS Classic at 3500 vertices (0.721 ms), which is a clear call-stack pressure artifact: a path of 3500 vertices produces a recursion depth of 3500, consuming significant stack memory and triggering GC pressure. BFS variants are unaffected as expected. This confirms the theoretical risk: DFS Classic on a path graph is one of the most vulnerable scenarios for stack overflow.

OVERALL RESULTS

The overall results reveal that algorithm performance is inseparable from graph topology. No single implementation dominates unconditionally across all graph types.

DFS Optimized is the strongest general-purpose variant, achieving the lowest or near-lowest times on RandomSparse, Tree, Cycle, Complete, Bipartite, DAG, Star, Path, and RandomDense. Eliminating recursion overhead while keeping a compact pre-allocated stack provides consistent dividends across nearly every topology.

BFS Classic performs competitively on dense graphs, and notably outperforms BFS Optimized on RandomDense at large sizes — an important counter-intuitive result that demonstrates that bit-manipulation cost can outweigh memory savings when neighbor lists are large.

BFS Optimized provides benefits primarily on memory-constrained or cache-sensitive workloads (sparse graphs, small sizes) but paradoxically lags behind BFS Classic on high-density graphs, confirming that the `BitArray` optimization trades throughput for memory footprint.

DFS Classic performs well on balanced trees and shallow graphs but degrades badly on linear chains (Path, Cycle) and deep structures, producing measurable spikes and risking stack overflow at production scale.

Grid graphs stand out as the most expensive topology, with times reaching seconds at large sizes regardless of algorithm, driven by cache-hostile traversal patterns inherent to 2D lattice access.

CONCLUSION

This empirical study of BFS and DFS traversal across ten graph topologies yields several profound insights that extend well beyond simple Big-O analysis.

While all four variants share the same $O(V + E)$ theoretical complexity, their real-world performance diverges dramatically depending on graph density, topology, and implementation strategy. The most critical findings are:

- **Recursion is a liability at scale:** DFS Classic produced measurable performance spikes on Cycle (1000 vertices), Path (3500 vertices), and RandomSparse (1750 vertices) graphs. The iterative DFS Optimized eliminated every single one of these anomalies, delivering 2–10× speedups in affected scenarios. In production systems where graph depth is unbounded, recursive DFS must be avoided entirely.

- **Memory optimization requires density awareness:** BFS Optimized's `BitArray` visited structure reduces memory consumption by 8× over `bool[]`, but this gain becomes a penalty on dense graphs. When each vertex has hundreds of neighbors, the repeated bit-manipulation (shifting and masking) in `BitArray.Get()` accumulates enough CPU cycles to make BFS Optimized slower than BFS Classic on RandomDense and Bipartite at large sizes. True optimization must account for the operational density of the target data.

- **Graph topology dominates over algorithm choice:** The Grid graph took over 4000 ms regardless of algorithm, while Star and Path graphs remained under 1 ms at equivalent vertex counts. The structural access pattern — not the traversal strategy — is the primary performance determinant in cache-sensitive environments. No algorithmic optimization can compensate for fundamentally cache-hostile memory access patterns imposed by a 2D lattice.

- **Pre-allocation is universally beneficial:** Both optimized variants pre-allocate their core data structures (`int[]` arrays) to the maximum possible size before traversal begins. This single change eliminates GC-triggering dynamic resizing events that the standard library collections (`Queue<int>`, call stack) incur during traversal, providing a consistent baseline speedup on every topology tested.

In conclusion, for general-purpose graph traversal, DFS Optimized is the recommended implementation. It is the fastest or near-fastest on 9 of the 10 tested topologies, is immune to stack overflow, and scales predictably. BFS Classic is the preferred BFS variant on dense graphs; BFS Optimized is better suited to sparse, memory-constrained contexts. DFS Classic should be restricted to shallow graphs where call depth is provably bounded.

REFERENCES

[1] Moisei, D. GitHub Repository. https://github.com/Moisei-D/Algorithm-Analysis

[2] GeeksforGeeks. (2024). *Breadth First Search or BFS for a Graph.* Retrieved March 2026, from https://www.geeksforgeeks.org/breadth-first-search-or-bfs-for-a-graph/

[3] GeeksforGeeks. (2024). *Depth First Search or DFS for a Graph.* Retrieved March 2026, from https://www.geeksforgeeks.org/depth-first-search-or-dfs-for-a-graph/